

pyenlight

Practical User Guide

Alexandros Aslanidis

July 2026

This guide is focused on using the library in practice. It explains how to install the package, define a scenario, run the PHY, MAC, and energy layers, and save results

Department of Informatics and Telematics
Harokopio University of Athens



1 Introduction

pyenlight is a Python-based cross-layer simulation framework for indoor optical and hybrid optical/RF IoT networks. The library jointly evaluates the physical layer, medium-access control (MAC), sensor-node energy consumption, optical energy harvesting, and battery lifetime.

The physical-layer model supports visible-light communication (VLC) downlinks and infrared (IR) or radio-frequency (RF) uplinks. Optical propagation may include line-of-sight (LoS), diffuse, and reconfigurable intelligent surface (RIS)-assisted components. Artificial lighting, sunlight entering through windows, and human blockage can also be included.

Sensor nodes may employ either photodiode (PD) or photovoltaic (PV) optical receivers. The PD model includes the transimpedance amplifier (TIA), the TIA noise, and optical shot noise [1]. The PV model is based on a five-parameter equivalent circuit and accounts for the electrical operating point, receiver bandwidth, noise, communication signal, and harvested power [2].

The MAC layer supports slotted and unslotted CSMA/CA, acknowledgements, backoff, clear-channel assessment (CCA), retransmissions, collisions, hidden nodes, physical-link failures, and optional busy-tone multiple access (BTMA) [3]. The MAC output is passed to the energy model so that the energy consumed during Tx, Rx, CCA, backoff, waiting, processing, sensing, initialization, and sleep is evaluated for each sensor node.

The main simulation workflow is:

1. Define the room, nodes, hardware, communication, MAC, and energy parameters in a design dictionary.
2. Create a `PhyNet` object to evaluate the optical and RF physical layers.
3. Export the physical-layer results using `PhyResultsDT0`.
4. Create an `EnergyManager` object (with an option for MAC simulation).
5. Save the per-node physical-layer, MAC, energy, harvesting, and battery results.

The overall simulation flow is

Design Dictionary → `PhyNet` → `PhyResultsDT0` → `EnergyManager`.

The library is intended for studies where the communication, propagation, hardware, and energy models must remain mutually consistent. Typical applications include the comparison of IR and RF uplinks based on the application requirements, PD and PV receivers, blockage conditions, RIS deployment, BTMA operation, network density, packet-delivery performance, and energy-neutral operation for GreenIoT.

2 Installation and Project Setup

2.1 Create a Virtual Environment

Open a terminal and navigate to the directory where the project will be stored. We strongly recommend to create a Python virtual environment using:

```
python3 -m venv pyenlight-env
```

Activate the environment using:

- **Linux or macOS:**

```
source pyenlight-env/bin/activate
```

- **Windows PowerShell:**

```
pyenlight-env\Scripts\Activate.ps1
```

- **Windows Command Prompt:**

```
pyenlight-env\Scripts\activate
```

After activation, the environment name should appear at the beginning of the terminal prompt.

2.2 Clone the Repository

Navigate to the directory where the repository will be stored and clone it using:

```
git clone https://github.com/AAslan94/enlight_iot.git
```

Then, enter the project directory:

```
cd enlight_iot
```

2.3 Install the Library

Upgrade pip and install the library in editable mode:

```
python -m pip install --upgrade pip  
python -m pip install -e .
```

The editable installation is recommended during development because changes made to the source code are immediately available without reinstalling the package.

The main dependencies include:

- numpy,
- scipy,
- matplotlib,
- simpy,
- pandas.

If required, they can be installed manually using:

```
python -m pip install numpy scipy matplotlib simpy pandas
```

2.4 Verify the Installation

Verify that **pyenlight** can be imported using:

```
python -c "import enlight_iot; print(enlight_iot.__file__)"
```

The printed path should point to the local project directory.

2.5 Run an Example Scenario

An existing scenario can be executed using, for example:

```
python run_A1.py
```

The corresponding scenario parameters are defined in:

```
design_A1.py
```

The design file specifies the room, nodes, optical and RF devices, communication parameters, MAC settings, and energy model. The run file loads the design, executes the simulation, and saves or plots the results.

2.6 Recommended Project Structure

For reproducible simulations, it is recommended to keep the scenario definition, execution script, results, and figures in separate files and directories:

```
experiment_name/  
|-- design_experiment.py  
|-- run_experiment.py  
|-- results/  
|   |-- experiment_metadata.json  
|   |-- experiment_phy_matrices.npz  
|   |-- experiment_phy_telemetry.npz  
|   '-- experiment_results.csv  
'-- plots/
```

The files are used as follows:

- `design_experiment.py` defines the room, nodes, optical and RF devices, protocol, MAC, and energy parameters.
- `run_experiment.py` loads the design, executes the PHY, MAC, and energy models, and saves the results.
- `experiment_metadata.json` stores the scenario name, simulation settings, random seeds, and other information required for reproducibility.
- `experiment_phy_matrices.npz` stores detailed PHY results, such as channel gains, received powers, and SNR components.
- `experiment_phy_telemetry.npz` stores the compact PHY results passed to the MAC and energy layers.
- `experiment_results.csv` stores the final per-node PHY, MAC, energy, harvesting, and battery results.
- The `plots` directory stores the generated figures.

2.7 Deactivate the Environment

After completing the simulation, deactivate the virtual environment using:

```
deactivate
```

3 Starting from an Existing Scenario

The recommended way to create a new simulation is to start from an existing validated scenario and modify only the parameters required for the new experiment.

Each scenario is normally organised using two Python files:

- a *design file*, which contains the simulation configuration; and
- a *run file*, which loads the design, executes the simulation, and processes the results.

For example, Scenario A1 is defined using:

```
design_A1.py  
run_A1.py
```

The design file specifies the room, nodes, optical and RF devices, communication settings, MAC parameters, and energy model. The run file contains the simulation workflow, including the execution of the PHY, MAC, and energy layers, result storage, and plotting.

3.1 Copy an Existing Scenario

To create a new scenario, copy both the design and run files:

```
cp design_A1.py design_example.py  
cp run_A1.py run_example.py
```

On Windows, the files may be copied using File Explorer or:

```
copy design_A1.py design_example.py  
copy run_A1.py run_example.py
```

The import statement inside the new run file must then be updated. For example:

```
from design_example import design
```

The imported variable name must match the design dictionary defined in the corresponding design file.

3.2 Design File

The design file defines the simulation scenario using a hierarchical Python dictionary. A simplified structure is:

```
design = {  
    "environment": {...},  
    "nodes": {...},  
    "PV_circuit": {...},  
    "TIA": {...},  
    "protocol": {...},  
    "energy_profile": {...},  
    "MPP": {...},  
}
```

The main sections are:

- **environment**: room dimensions, surface reflectivity, wall discretisation, windows, RISs, and human blockers;
- **nodes**: master nodes, sensor nodes, and ambient optical sources;
- **PV_circuit**: parameters of the PV equivalent circuit;
- **TIA**: transimpedance-amplifier parameters for PD-based receivers;
- **protocol**: packet lengths, data rates, cycle period, and communication parameters;
- **energy_profile**: hardware currents, task durations, battery parameters, driver models, and MAC settings;
- **MPP**: maximum-power-point and energy-conversion parameters.

The design dictionary does not need to explicitly define every supported parameter. When a value is omitted, the library uses the corresponding fallback value from the `EnLightConfig` object, defined in:

```
enlight_iot/core/config.py
```

The configuration object contains default physical, spectral, environmental, communication, hardware, and device parameters. Therefore, a scenario may define only the values that differ from the defaults.

Values explicitly defined in the design dictionary generally override the corresponding values in `EnLightConfig`. This allows common defaults to be retained while individual scenarios modify only the required parameters.

A configuration object is created using:

```
from enlight_iot.core.config import EnLightConfig

config = EnLightConfig()
```

Its values may also be modified before the simulation. For example:

```
config.env.bounces = 4
config.comm.T_cycle = 60.0
config.hardware.battery_capacity_mAh = 500
```

The complete design structure and the available parameters are provided in the appendix.

3.3 Run File

The run file imports the design dictionary and executes the required simulation stages. A typical structure is:

```
from enlight_iot.core.config import EnLightConfig
from enlight_iot.network.orchestrator import PhyNet
from enlight_iot.hardware.energy import EnergyManager
from design_example import design

config = EnLightConfig()

phy = PhyNet(
    design,
    budget_run=False,
    config=config,
    btma_mode=True,
)

phy_data = phy.export_energy_telemetry()
```

```
energy = EnergyManager(  
    phy_data=phy_data,  
    design=design,  
    config=config,  
    MAC=True,  
    btma_mode=True,  
    MAC_mode="unslotted",  
)  
  
results = energy.get_results_df()
```

Creating the `PhyNet` object executes the physical-layer simulation. This includes the construction of the room and nodes, channel-gain calculation, receiver-noise evaluation, and computation of the optical and RF communication metrics.

The `export_energy_telemetry()` method creates a compact physical-layer output containing the values required by the MAC and energy layers.

Creating the `EnergyManager` object evaluates the sensor-node energy consumption. When `MAC=True`, the MAC simulation is also executed, and its per-node Tx, Rx, CCA, backoff, waiting, and retransmission durations are included in the energy calculation.

Some execution-specific settings are not read from the design dictionary or from `EnLightConfig`. They are instead passed directly to the corresponding constructor. These include:

- `budget_run`, which enables the PHY link-budget calculation;
- `btma_mode`, which enables or disables BTMA-related behaviour;
- `MAC`, which enables or disables the MAC simulation; and
- `MAC_mode`, which selects the slotted or unslotted MAC implementation.

3.4 Recommended Workflow

When creating or modifying a scenario, the following procedure is recommended:

1. Copy an existing validated design and run file.
2. Rename both files according to the new scenario.
3. Update the design import inside the run file.
4. Modify only one group of parameters at a time.
5. Execute a small test scenario.
6. Inspect the dimensions and ranges of the PHY results.
7. Validate the LoS, diffuse, RIS, and blockage behaviour.
8. Enable the MAC and energy layers after validating the PHY.
9. Increase the number of nodes, wall resolution, simulation duration, and number of seeds only after the initial checks.
10. Save the final design, configuration, metadata, PHY outputs, and results.

4 Running the Physical-Layer Simulation

The physical-layer simulation is executed by creating a `PhyNet` object. The object receives the design dictionary, the configuration object, and the execution-specific PHY options.

A minimal PHY run is:

```
from enlight_iot.core.config import EnLightConfig
from enlight_iot.network.orchestrator import PhyNet
from design_example import design

config = EnLightConfig()

phy = PhyNet(
    design,
    budget_run=False,
    config=config,
    btma_mode=True,
)
```

Creating the `PhyNet` object immediately executes the physical-layer simulation, so it may take some time depending on the scenario

The main operations performed during its initialisation are:

1. construction of the room and its surfaces;
2. construction of the master, sensor, and ambient nodes;
3. calculation of the LoS optical gains;
4. calculation of the diffuse optical gains;
5. calculation of the RIS-assisted gains, when applicable;
6. calculation of the RF uplink gains;
7. evaluation of the ambient optical power;
8. calculation of the receiver bandwidth and noise;
9. calculation of the received optical and RF powers; and
10. calculation of the PHY communication metrics.

4.1 PhyNet Arguments

The main arguments of `PhyNet` are:

- `design`: the complete scenario dictionary;
- `budget_run`: enables or disables the link-budget calculation;
- `config`: the `EnLightConfig` object containing the fallback values;
- `btma_mode`: enables or disables the PHY calculations associated with BTMA operation.

A normal simulation with predefined Tx powers is performed using:

```
budget_run=False
```


When:

```
budget_run=True
```

the library performs a link-budget calculation. For a single master node, the sensor node transmitters are oriented towards the corresponding master node receiver. The minimum IR optical or RF transmit power required to satisfy the defined PHY target is then estimated for each sensor node. This mode is therefore useful for link-budget studies and for determining the minimum required uplink transmit power across the considered deployment.

4.2 Downlink Calculation

The downlink is implemented using VLC from the master node to the sensor nodes. The received signal may contain:

- a LoS component;
- a diffuse component;
- an RIS-assisted component
- ambient light contributions used in the receiver noise calculation.

The total downlink signal is evaluated using the propagation components and the receiver type assigned to each sensor node.

The optical receiver type of each sensor node is selected using the `rx_type` parameter:

```
rx_type = 0    # PD receiver
rx_type = 1    # PV receiver
```

Mixed deployments are supported by assigning a separate `rx_type` value to each sensor node.

For PD-based receivers, the model includes the TIA and the associated receiver noise [1]. For PV-based receivers, the model uses the PV equivalent circuit, the PV operating point, the receiver bandwidth, and the corresponding electrical noise [2].

4.3 Uplink Calculation

Each sensor node may use either an IR or RF uplink.

The uplink type is selected using:

```
uplink_type = 0    # IR uplink
uplink_type = 1    # RF uplink
```

For the IR uplink, the received signal may include LoS, diffuse, and RIS-assisted optical components.

For the RF uplink, the received power is calculated using the configured indoor RF propagation model.

Mixed sensor populations are supported by assigning a different `uplink_type` value to each sensor node.

4.4 Accessing PHY Results

The main PHY results are stored as attributes of the `PhyNet` object.

The downlink SNR can be accessed using:

```
phy.snr_d_dB
```

The uplink SNR can be accessed using:

```
phy.snr_u_dB
```

The sensor-to-sensor SNR, used for channel sensing and hidden-node evaluation, can be accessed using:

```
phy.snr_ss_dB
```

We recommend to inspect the corresponding array dimensions before further processing:

```
print("Downlink SNR shape:", phy.snr_d_dB.shape)
print("Uplink SNR shape:", phy.snr_u_dB.shape)
print("Sensor-to-sensor SNR shape:", phy.snr_ss_dB.shape)
```

For N sensor nodes, the downlink and uplink arrays normally contain one value per sensor. The sensor-to-sensor result is generally an $N \times N$ matrix.

4.5 Inspecting the Results

A simple inspection may be performed using:

```
print("Minimum DL SNR:", phy.snr_d_dB.min())
print("Maximum DL SNR:", phy.snr_d_dB.max())

print("Minimum UL SNR:", phy.snr_u_dB.min())
print("Maximum UL SNR:", phy.snr_u_dB.max())
```

The following checks are recommended before continuing to the MAC and energy layers:

- verify that the result arrays have the expected dimensions;
- verify that the SNR values are finite;
- verify that blocked links behave as expected;
- verify that links outside the receiver FOV have zero gain;
- verify that the RIS contribution disappears when the RIS is removed;
- verify that the received power changes consistently with the Tx power; and
- verify that the sensor ordering matches the design file.

4.6 Saving PHY Results

Detailed PHY results can be saved using:

```
phy.save_phy_state(
    "results/experiment_phy_matrices.npz"
)
```

This file contains detailed channel gains, received powers, SNR components, and other PHY arrays.

A compact set of PHY results can be exported using:

```
phy_data = phy.export_energy_telemetry()
```

The compact PHY output can then be saved using:

```
phy_data.save_npz(  
    "results/experiment_phy_telemetry.npz"  
)
```

The detailed PHY matrices and the compact telemetry serve different purposes. The detailed file is intended for PHY analysis, while the telemetry file contains the values required by the MAC and energy layers.

Saving the telemetry is recommended when several MAC or energy experiments use the same room, nodes, and PHY configuration. In that case, the computationally expensive PHY simulation does not need to be repeated for every MAC run.

5 Running the Energy Model

The energy model is implemented through the `EnergyManager` class. It combines the PHY results, the sensor-node hardware parameters, the communication settings, and, optionally, the MAC-layer output.

Before creating the `EnergyManager` object, the PHY results must be exported using:

```
phy_data = phy.export_energy_telemetry()
```

The returned `PhyResultsDTO` object contains the compact set of PHY parameters required by the energy and MAC layers, including:

- the number of sensor nodes;
- the uplink and downlink data rates;
- the receiver type of each sensor node;
- the uplink type of each sensor node;
- the optical and RF transmit powers;
- the downlink, uplink, and sensor-to-sensor SNR values;
- the PV voltage and current operating points.

5.1 Energy Model without MAC

The energy consumption can be evaluated without considering MAC-layer overhead:

```
energy_no_mac = EnergyManager(  
    phy_data=phy_data,  
    design=design,  
    config=config,  
    MAC=False,  
    btma_mode=True,  
    MAC_mode="unslotted",  
)
```

When `MAC=False`, the sensor-node cycle is evaluated using the configured task and communication durations. Backoff, CCA, retransmissions, collisions, and ACK failures are not included. Naturally, also the `btma_mode` and `MAC_mode` options do not matter in this case.

This mode provides a useful baseline for evaluating the energy required by:

- node initialisation;
- sensing;
- processing;
- one uplink transmission;
- one downlink reception;
- waiting or turnaround; and
- sleep.

It is particularly useful for comparing:

- IR and RF uplinks;
- PD and PV optical receivers;
- different Tx powers;
- different packet lengths and data rates; and
- different hardware current profiles.

5.2 Energy Model with MAC

To include MAC-layer operation, create the `EnergyManager` object using:

```
energy_mac = EnergyManager(  
    phy_data=phy_data,  
    design=design,  
    config=config,  
    MAC=True,  
    btma_mode=True,  
    MAC_mode="unslotted",  
)
```

When `MAC=True`, the MAC simulation is executed and the per-node state durations are passed to the energy model.

The calculated energy then includes the effects of:

- backoff;
- CCA;
- multiple Tx attempts;
- retransmissions;
- collisions;
- uplink PHY failures;
- downlink or ACK failures;
- channel-access failures;
- BTMA operation, when enabled.

The MAC-aware energy model therefore provides a more realistic estimate of the sensor-node consumption under contention and imperfect link conditions.

5.3 EnergyManager Arguments

The main arguments of `EnergyManager` are:

- `phy_data`: the compact PHY results exported from `PhyNet`;
- `design`: the scenario design dictionary;
- `config`: the `EnLightConfig` object containing the fallback values;
- `MAC`: enables or disables the MAC simulation, `True` or `False`;
- `btma_mode`: enables or disables BTMA-related operation, `True` or `False` ;
- `MAC_mode`: selects the MAC implementation, `"unslotted"` or `"slotted"`.

5.4 Modeled Energy States

The sensor-node cycle may include the following states:

1. initialisation;
2. sensing;
3. processing;
4. backoff;
5. CCA;
6. uplink transmission;
7. turnaround or waiting;
8. downlink or ACK reception; and
9. sleep.

The energy consumed in each state is determined from the corresponding current, operating voltage, and state duration.

For PD-based receivers, the downlink reception current includes the TIA current. For PV-based receivers, the TIA current is not included.

5.5 Accessing the Energy Results

The complete per-node results can be obtained as a Pandas `DataFrame`:

```
results = energy_mac.get_results_df()
```

The first rows may be inspected using:

```
print(results.head())
```

The available columns depend on whether the MAC layer is enabled. Typical outputs include:

- cycle energy;
- daily consumed energy;
- daily harvested energy;

- daily net energy;
- estimated battery lifetime;
- MAC packet delivery ratio;
- CCA attempts;
- backoff slots;
- retransmissions;
- Tx attempts; and
- channel-access failures.

5.6 Saving the Results

The results can be saved directly to a CSV file:

```
energy_mac.save_csv(  
    "results/experiment_results.csv"  
)
```

6 Running the MAC Simulation

The MAC layer is implemented in `enlight_iot/network/mac.py` using SimPy. It supports slotted and unslotted CSMA/CA, periodic or stochastic traffic, acknowledgements, backoff, CCA, retransmissions, collisions, hidden nodes, PHY link failures, and optional BTMA.

The MAC simulation may be executed automatically through `EnergyManager` or directly using the functions provided in `mac.py`.

6.1 MAC Execution through EnergyManager

When:

`MAC=True`

the `EnergyManager` prepares the PHY-dependent MAC inputs and executes the MAC simulation.

For RF sensor nodes, the compact PHY telemetry contains:

- the binary RF uplink availability of each RF sensor node, determined from whether its best received power margin relative to the master node sensitivity is non-negative;
- the RF hidden-node matrix, indicating which RF sensor nodes cannot detect each other's transmissions.

For IR sensor nodes, the corresponding indicators are derived from the optical PHY results. In particular:

- IR uplink availability is determined by comparing the IR uplink SNR with the configured SNR threshold; and
- the IR hidden-node matrix is determined by comparing the sensor-to-sensor IR SNR values with the same threshold.

The complete mixed-network MAC inputs are then constructed by combining the RF and IR indicators according to the uplink type of each sensor node.

6.2 MAC Modes

The MAC implementation is selected using the `MAC_mode` argument:

```
MAC_mode="unslotted"
```

or:

```
MAC_mode="slotted"
```

In unslotted mode, each sensor node performs backoff and CCA directly in simulation time without alignment to a common slot boundary.

In slotted mode, the nodes align their backoff and CCA operations to a shared slot clock, similar to the MAC contention access period (CAP) defined in 802.15.7 and 802.15.4.

For the current simulation studies, the unslotted mode is generally used.

6.3 BTMA Operation

BTMA is enabled using:

```
btma_mode=True
```

When BTMA is enabled, sensor nodes may detect activity at the master node through the busy tone, even when they cannot directly detect another sensor-node transmission.

The busy-tone hidden mask identifies sensor nodes that cannot detect the busy tone. These nodes may remain hidden and may transmit while another uplink transmission is already active.

BTMA is disabled using:

```
btma_mode=False
```

In this case, channel sensing is based on the sensor-to-sensor link information.

6.4 Main MAC Configuration Parameters

The MAC configuration is normally provided inside the design dictionary:

```
design["energy_profile"]["MAC"] = {  
    "sim_time_us": 3000e6,  
    "n_seeds": 150,  
    "SNR_THRESHOLD_dB": 8.5,  
    "BUSY_TONE_THRESHOLD_dB": 8.5,  
    "log": True,  
    "debug": False,  
}
```

The main parameters are:

- `sim_time_us`: simulation duration in microseconds;
- `n_seeds`: number of independent MAC realisations;
- `SNR_THRESHOLD_dB`: threshold used to determine optical uplink availability and IR hidden-node conditions;

- `BUSY_TONE_THRESHOLD_dB`: threshold used to determine whether a sensor node can detect the busy tone;
- `log`: enables summary output; and
- `debug`: enables detailed event-level output.

During debugging, a small number of seeds and a short simulation duration should be used. The final simulation should use a sufficiently large number of seeds and duration to reduce statistical variation.

6.5 Direct MAC Execution

A single MAC simulation may be executed using `run_sim()`:

```
from enlight_iot.network.mac import run_sim

stats, params = run_sim(
    n_nodes=20,
    mean_iat_us=60e6,
    mode="unslotted",
    traffic_type="periodic",
    seed=1,
    sim_time_us=3000e6,
    data_rate_bps=10e3,
    symbol_rate_sym_s=10e3,
    payload_bytes=128,
    ack_bytes=16,
    hidden_node_mask=None,
    phy_pdr_up=None,
    phy_pdr_down=None,
    bt_hidden_mask=None,
    btma_mode=True,
    debug=False,
)
```

The main inputs are:

- the number of nodes;
- packet-generation interval;
- MAC mode;
- traffic model;
- random seed;
- simulation duration;
- data and symbol rates;
- payload and ACK lengths;
- hidden-node matrix;
- uplink and downlink PHY availability;
- busy-tone hidden mask; and
- BTMA state.

6.6 Multiple MAC Realisations

Multiple independent realisations may be executed using `call_MAC()`:

```
from enlight_iot.network.mac import call_MAC

mean, std, runs, per_node, params = call_MAC(
    nodes=20,
    period=60e6,
    mode="unslotted",
    traffic_type="periodic",
    n_seeds=150,
    sim_time_us=3000e6,
    data_rate_bps=10e3,
    symbol_rate_sym_s=10e3,
    payload_bytes=128,
    ack_bytes=16,
    hidden_node_mask=hidden_node_mask,
    bt_hidden_mask=bt_hidden_mask,
    btma_mode=True,
    phy_pdr_up=phy_pdr_up,
    phy_pdr_down=phy_pdr_down,
    log=True,
    debug=False,
)
```

The function returns:

- the mean MAC metrics;
- the corresponding standard deviations;
- the results from all seeds;
- the aggregated per-node results; and
- the MAC timing and frame parameters.

6.7 Main MAC Outputs

Typical MAC outputs include:

- generated packets;
- successfully delivered packets;
- packet delivery ratio;
- collisions;
- uplink PHY failures;
- downlink or ACK failures;
- channel-access failures;
- CCA attempts;
- backoff slots;
- Tx attempts;

- retransmissions;
- packet delay;
- Tx, Rx, CCA, waiting, and backoff times; and
- per-node state durations used by the energy model.

The channel-access failure metric represents packets discarded after the maximum permitted CCA attempts are exhausted.

7 Interpreting the Main Results

The simulation produces PHY, MAC, energy-consumption, energy-harvesting, and battery-related results. The available outputs depend on the selected node types and on whether the MAC layer is enabled.

The exact columns returned by the energy model can be inspected using:

```
print(results.columns)
```

7.1 PHY Results

The main PHY results include:

- `snr_d_dB`: downlink SNR for each sensor node;
- `snr_u_dB`: uplink SNR for each sensor node;
- `snr_ss_dB`: sensor-to-sensor SNR matrix used for channel sensing and hidden-node evaluation;
- LoS, diffuse, and RIS-assisted channel gains;
- received optical or RF powers;
- receiver-noise components; and
- PV operating voltage, current, and bandwidth, when PV receivers are considered.

For N_{SN} sensor nodes, the downlink and uplink arrays normally contain one value per sensor node. The sensor-to-sensor result is generally an $N_{SN} \times N_{SN}$ matrix.

The total SNR should be interpreted together with the individual LoS, diffuse, and RIS-assisted components. A high total SNR may result from a strong direct path, reflected power, RIS assistance, low receiver noise, or a combination of these effects.

7.2 MAC Results

When the MAC layer is enabled, typical outputs include:

- generated packets;
- successfully delivered packets;
- MAC packet delivery ratio;
- collisions;
- uplink PHY failures;
- downlink or ACK failures;

- channel-access failures;
- CCA attempts;
- backoff slots;
- Tx attempts;
- retransmissions;
- packet delay; and
- per-node MAC state durations.

The MAC packet delivery ratio should be interpreted together with the individual packet-loss mechanisms. A reduced PDR may result from collisions, uplink PHY failure, ACK failure, exhaustion of the retransmission limit, or exhaustion of the permitted CCA attempts.

A low collision rate does not necessarily indicate efficient operation. For example, packets may be discarded after repeated CCA failures before transmission. This reduces the number of collisions, but also reduces packet delivery.

7.3 Energy Results

Typical energy outputs include:

- cycle energy;
- daily consumed energy;
- daily harvested energy;
- daily net energy;
- estimated battery lifetime; and
- energy per delivered packet.

The total daily network energy may be obtained by summing the per-node values:

```
network_energy = results[
    "E_day_consumed"
].sum()
```

The exact column name should be verified from the returned `DataFrame`.

The energy per delivered packet is useful when comparing cases with different packet-delivery ratios. However, it should be considered together with the total network energy and the number of delivered packets.

A lower cycle energy does not always indicate improved operation. Under severe contention, packets may be discarded after exhausting the allowed CCA attempts. In this case, the energy-intensive Tx and ACK Rx states are avoided, reducing the average cycle energy while also reducing packet delivery.

7.4 Harvesting and Battery Results

The daily net energy is calculated from the difference between the harvested and consumed energy.

A positive daily net energy indicates that the modeled harvested energy is equal to or greater than the modeled daily consumption.

This result should not be interpreted as an infinite practical battery lifetime without qualification. The current model does not fully include battery aging, self-discharge, finite charge capacity, temperature dependence, or long-term degradation.

Therefore, a non-depleting battery result represents idealised energy-neutral operation under the modeled conditions.

7.5 Comparison of Different Cases

When comparing different scenarios, the following parameters should remain unchanged unless they are the subject of the comparison:

- room geometry;
- sensor positions;
- node orientations;
- Tx powers;
- packet lengths;
- data rates;
- traffic period;
- MAC simulation duration;
- number of seeds;
- random seeds;
- PHY thresholds;
- hardware-current values; and
- harvesting duration.

Only the investigated parameter should normally change between the compared cases.

8 Complete Usage Template

The following example presents the complete cross-layer workflow. The PHY simulation is first executed and its results are exported through a `PhyResultsDTO` object. This object is then passed to `EnergyManager`.

When `MAC=True`, creating the `EnergyManager` object automatically executes the MAC simulation. The resulting per-node MAC state durations are subsequently used in the energy calculation.

```
from pathlib import Path

from enlight_iot.core.config import EnLightConfig
from enlight_iot.network.orchestrator import PhyNet
from enlight_iot.hardware.energy import EnergyManager

from design_example import design

# -----
# Output directory
# -----

OUT = Path("results")
OUT.mkdir(parents=True, exist_ok=True)

# -----
# Configuration
# -----

config = EnLightConfig()

# -----
# 1. Physical-layer simulation
# -----

phy = PhyNet(
    design,
    budget_run=False,
    config=config,
    btma_mode=True,
)

phy.save_phy_state(
    OUT / "experiment_phy_matrices.npz"
)

# -----
# 2. Export the PHY-to-MAC/energy interface
# -----

phy_data = phy.export_energy_telemetry()

phy_data.save_npz(
    OUT / "experiment_phy_telemetry.npz"
)
```

```

# -----
# 3. Energy calculation without MAC
# -----

energy_no_mac = EnergyManager(
    phy_data=phy_data,
    design=design,
    config=config,
    MAC=False,
    btma_mode=True,
    MAC_mode="unslotted",
)

energy_no_mac.save_csv(
    OUT / "experiment_no_mac.csv"
)

# -----
# 4. MAC simulation and MAC-aware energy calculation
# -----

energy_mac = EnergyManager(
    phy_data=phy_data,
    design=design,
    config=config,
    MAC=True,
    btma_mode=True,
    MAC_mode="unslotted",
)

energy_mac.save_csv(
    OUT / "experiment_mac.csv"
)

# -----
# 5. Access the results
# -----

results_no_mac = energy_no_mac.get_results_df()
results_mac = energy_mac.get_results_df()

print(results_no_mac.head())
print(results_mac.head())

```

The MAC simulation is triggered by:

```
MAC=True
```

A Indicative Complete Design

The following example contains the parameters that may currently be provided through the design dictionary. The parameter names and nesting follow the implementation of **pyenlight**.

The `positions` field is required for every node group that is included in the design. Most other values are optional

and fall back to the corresponding values in EnLightConfig.

```
import numpy as np

from enlight_iot.core.utils import generate_grid

design = {

    # =====
    # Environment
    # =====

    "environment": {

        # Room dimensions [length, width, height] in m
        "dimensions": [5.0, 5.0, 3.0],

        # Number of patches used along each surface axis
        "wall_resolution": [20, 20],

        # Surface reflectivities
        "reflectivity": {
            "floor": 0.2,
            "ceiling": 0.6,
            "walls": 0.8,
        },

        # Optional windows and RIS surfaces
        "special_surfaces": [

            {

                # Supported values: "window" or "RIS"
                "type": "window",

                # User-defined surface name
                "name": "West Window",

                # Surface centre [x, y, z] in m
                "center": [0.0, 2.5, 1.5],

                # Surface dimensions in m
                "dims": [1.0, 1.0],

                # Constant coordinate:
                # 0 -> x, 1 -> y, 2 -> z
                "const_axis": 0,

                # Surface discretisation
                "resolution": [20, 20],

                # Surface normal
                "normal": [1.0, 0.0, 0.0],

                # Surface reflectivity
                "reflectivity": 0.05,

            },

        ],

    },

}
```

```

    {
        "type": "RIS",
        "name": "East RIS",
        "center": [5.0, 2.5, 1.5],
        "dims": [1.0, 1.0],
        "const_axis": 0,
        "resolution": [20, 20],
        "normal": [-1.0, 0.0, 0.0],
        "reflectivity": 0.9,
    },
],

# Optional human blockers
"blockers": {

    # Blocker floor-plane centres [x, y, z]
    "positions": np.array([
        [2.5, 2.5, 0.0],
        [3.5, 2.0, 0.0],
    ]),

    # Blocker radius in m.
    # May be a scalar or a compatible array.
    "radius": 0.3,

    # Blocker height in m.
    # May be a scalar or a compatible array.
    "height": 1.7,
},

},

# =====
# Nodes
# =====

"nodes": {

    # -----
    # Master nodes
    # -----

    "masters": {

        # Required master-node positions
        "positions": np.array([
            [2.5, 2.5, 3.0],
        ]),

        # Optical transmitter normal
        "nT": [0.0, 0.0, -1.0],

        # Optical receiver normal
        "nR": [0.0, 0.0, -1.0],

        # Optical receiver area in m^2
        "rx_area": 1e-4,
    },
},

```



```

    # Lambertian order of the VLC transmitter
    "m": 1,

    # Optical receiver FOV in rad
    "FOV": np.pi / 2,

    # VLC downlink optical transmit power in W
    "tx_power": 1.0,

    # RF receiver sensitivity in dBm
    "sensitivity": -100.0,

    # True enables the IR optical filter at the
    # master-node receiver
    "IR_pass_filter": True,
},

# -----
# Sensor nodes
# -----

"sensors": {

    # Required sensor-node positions
    "positions": generate_grid(
        0.2, 4.8,
        0.2, 4.8,
        0.0,
        10, 10,
        False,
    ),

    # IR transmitter normal
    "nT": [0.0, 0.0, 1.0],

    # Downlink optical receiver normal
    "nR": [0.0, 0.0, 1.0],

    # Optical receiver area in m^2
    "rx_area": 1e-4,

    # Lambertian order of the IR transmitter
    "m": 1,

    # Optical receiver FOV in rad
    "FOV": np.pi / 2,

    # Receiver type:
    # 0 -> PD
    # 1 -> PV
    #
    # May be a scalar or one value per sensor.
    "rx_type": 0,

    # Uplink type:
    # 0 -> IR

```

```

    # 1 -> RF
    #
    # May be a scalar or one value per sensor.
    "uplink_type": 0,

    # IR uplink optical transmit power in W
    "IR_tx_power": 15e-3,

    # RF uplink transmit power in dBm
    "RF_tx_power": -20.0,

    # RF receiver sensitivity used for
    # sensor-to-sensor RF sensing
    "sensitivity": -100.0,

    # True enables the VLC optical filter at
    # PD-based sensor receivers
    "VLC_pass_filter": True,
},

# -----
# Optional ambient optical sources
# -----

"ambient_nodes": {

    # Required ambient-source positions
    "positions": np.array([
        [1.25, 1.25, 3.0],
        [3.75, 1.25, 3.0],
        [1.25, 3.75, 3.0],
        [3.75, 3.75, 3.0],
    ]),

    # Ambient-source transmitter normals
    "nT": [0.0, 0.0, -1.0],

    # Lambertian order
    "m": 3,

    # Optical transmit power in W
    "tx_power": 5.0,
},
},

# =====
# Communication parameters
# =====

"energy_profile": {

    # Sensor-node operation period in s
    "T_cycle": 60.0,

    # -----
    # Communication rates and bandwidth factors

```

```

# -----

"communication": {

    # VLC downlink bit rate in bit/s
    "Rb_down": 10e3,

    # Downlink bandwidth factor
    "n_sp_d": 0.4,

    # Uplink bit rate in bit/s.
    #
    # May be a scalar or one value per sensor.
    # If omitted, IR and RF rates are selected
    # from EnLightConfig according to uplink_type.
    "Rb_up": 10e3,

    # Uplink bandwidth factor.
    #
    # May be a scalar or one value per sensor.
    "n_sp_u": 0.4,
},

# -----
# Hardware parameters
# -----

"hardware": {

    # MCU clock frequency in Hz
    "f_mcu": 16e6,

    # Sensing frequency in Hz
    "f_s": 1e3,

    # Sensor-node operating voltage in V
    "voltage": 3.3,
},

# -----
# Task parameters
# -----

"tasks": {

    # Number of acquired samples per cycle
    "N_s_up": 100,

    # Number of processing cycles per cycle
    "N_c_up": 1e3,

    # Uplink packet length in bit
    "L_up_bits": 1024,

    # Downlink or ACK packet length in bit
    "L_dw_bits": 128,

```

```

},

# -----
# State currents
# -----

# MCU active current in A
"I_mcu": 2.73e-3,

# ADC current in A
"I_adc": 0.7e-3,

# External sensor current in A
"I_ext": 1.0e-3,

# Sleep current in A
"I_sleep": 2e-6,

# Wake-up current in A
"I_wake": 1e-3,

# TIA current in A
"I_tia": 0.7e-3,

# -----
# Battery parameters
# -----

"battery": {

    # Battery capacity in mAh
    "battery_capacity_mAh": 500,

    # Nominal battery voltage in V
    "V_batt": 3.6,

    # Initial state of charge
    "initial_soc": 1.0,
},

# -----
# IR transmitter-current model
# -----

"IRDriver": {

    # Maximum driver current in A
    "imax": 100e-3,

    # Minimum driver current in A
    "imin": 0.0,

    # Polynomial mapping current to optical power
    "pol": np.array([
        1.353e-1,

```

```

        1.868e-1,
        -1.017e-4,
    ]),

    # Inverse polynomial mapping optical power
    # to driver current
    "polinv": np.array([
        -1.740e1,
        5.329,
        5.618e-4,
    ]),
},

# -----
# RF transmitter-current model
# -----

"RFDriver": {

    # Minimum supported RF transmit power in dBm
    "p_min": -20.0,

    # Maximum supported RF transmit power in dBm
    "p_max": 5.0,

    # Polynomial mapping RF power in dBm to
    # current consumption in mA
    "pol": np.array([
        0.24,
        8.8,
    ]),
},

# -----
# MAC simulation parameters
# -----

"MAC": {

    # MAC simulation duration in us
    "sim_time_us": 3000e6,

    # Number of independent random seeds
    "n_seeds": 150,

    # SNR threshold used for optical links in dB
    "SNR_THRESHOLD_dB": 8.5,

    # Busy-tone detection threshold in dB
    "BUSY_TONE_THRESHOLD_dB": 8.5,

    # Print aggregate MAC output
    "log": True,

    # Print detailed event-level output
    "debug": False,

```

```

    },
},

# =====
# Protocol parameters
# =====

"protocol": {

    # Initialisation duration in s
    "t_init": 5e-3,

    # Turnaround or waiting duration in s
    "t_wait": 1e-3,

    # Daily harvesting duration in h
    #
    # May be a scalar or values compatible with the
    # number of PV-equipped sensor nodes.
    "harvesting_hours": 5.0,
},

# =====
# Maximum-power-point conversion
# =====

"MPP": {

    # Energy-conversion efficiency
    "mpp_eff": 0.8,
},

# =====
# Transimpedance amplifier
# =====

"TIA": {

    # Feedback resistance in ohm
    "RF": 1e6,

    # Input-referred voltage-noise density
    "Vn": 15e-9,

    # Input-referred current-noise density
    "In": 400e-15,

    # Voltage-noise corner frequency in Hz
    "fncV": 1e3,

    # Current-noise corner frequency in Hz
    "fncI": 1e3,

    # TIA temperature in K
    "temperature": 300.0,

```

```

},

# =====
# PV equivalent-circuit parameters
# =====

"PV_circuit": {

    # PV active area in m^2
    "A": 1e-4,

    # Diode ideality factor
    "n": 1.6,

    # Series resistance in ohm
    "Rs": 1.0,

    # Shunt resistance in ohm
    "Rsh": 1000.0,

    # Open-circuit voltage under the reference
    # condition in V
    "Voc": 0.64,

    # Short-circuit current density under the
    # reference condition in A/cm^2
    "Jsc": 35e-3,

    # Output inductor in H
    "Lo": 1e-6,

    # Communication coupling capacitor in F
    "Co": 1e-6,

    # Communication load resistance in ohm
    "Rc": 10.0,

    # Acceptor concentration
    "Na": 1.0e22,

    # Donor concentration
    "Nd": 1.0e25,

    # Diffusion length in m
    "L": 300e-6,

    # Relative permittivity
    "er": 11.68,

    # Intrinsic carrier concentration
    "ni": 1.0e16,
},
}

```

A.1 Parameters Defined outside the Design Dictionary

Some parameters are not currently read from the design dictionary. They must be modified through `EnLightConfig` or passed directly to a class constructor.

For example, the number of diffuse reflections is taken from:

```
config.env.bounces
```

and not from:

```
design["environment"]["bounces"]
```

A configuration object may be modified as follows:

```
from enlight_iot import EnLightConfig

config = EnLightConfig()

# Number of diffuse reflections
config.env.bounces = 4

# Default room and optical parameters
config.env.wall_resolution = (20, 20)
config.env.reflectivity = 0.6
config.env.rx_area = 1e-4
config.env.m = 1
config.env.fov = np.pi / 2

# Default communication parameters
config.comm.bit_rate_up_ir = 10e3
config.comm.bit_rate_up_rf = 250e3
config.comm.bit_rate_dw = 10e3

# RF propagation model
config.devices.rf_channel = {
    "n": 1.46,
    "pl_ref": 34.62,
    "k": 2.03,
    "f": 2.45,
    "sigma": 3.76,
    "sigma_factor": 2,
}
```

The following execution-specific options are passed directly to the constructors and are not included in the design dictionary:

```
phy = PhyNet(
    master_design_example,
    budget_run=False,
    btma_mode=True,
    config=config,
)

energy = EnergyManager(
    phy_data,
    master_design_example,
```



```
    config=config,  
    MAC=True,  
    btma_mode=True,  
    MAC_mode="unslotted",  
)
```

In particular:

- `budget_run` enables the PHY link-budget procedure;
- `MAC` enables the MAC simulation;
- `btma_mode` enables BTMA operation; and
- `MAC_mode` selects the slotted or unslotted CSMA/CA implementation.

References

- [1] Kamalakis, T., Ghassemlooy, Z., Zvanovec, S., and Alves, L. N. (2022a). Analysis and simulation of a hybrid visible-light/infrared optical wireless network for IoT applications. *J. Opt. Commun. Netw.*, 14(3):69–78.
- [2] Aslanidis, A., Kamalakis, T., Zvanovec, S., Zamorano-Illanes, R., and Ghassemlooy, Z. (2026). Area- efficient design and optimization of dual-function photovoltaic receivers for indoor optical IoT networks. *Optical Engineering*, 65(7):078103
- [3] Gupta, A. and Dixit, A. (2024). Mathematical analysis of busy tone in full-duplex optical mac for hidden node mitigation. *Computer Networks*, 255:110870